

Sampling, FFT, and the Pre-Lab 1 notebook

Fall 2026 · Prof. Ehsan Roohi

Lecture and guided Jupyter walkthrough

Learning goals

- Explain sample rate, sample interval, record length, and Nyquist frequency
- Run code and Markdown cells in a Jupyter notebook
- Interpret time histories, RMS values, and one-sided FFT spectra
- Recognize aliasing before collecting Lab 1 sound data

Connection to Lab 1

PRE-LAB

A known two-tone signal tests sampling and FFT ideas before the laboratory.

The code provides a controlled result that you can predict.

LABORATORY

The Moku:Go records real microphone signals.

You will compare waveforms, spectra, and replayed sound at several sample rates.

Jupyter notebook structure

MARKDOWN CELLS

Instructions, equations, and response areas.

Double-click to edit. Use Shift+Enter to render and move forward.

CODE CELLS

Python commands that define variables, calculate results, and make figures.

Run from top to bottom with Shift+Enter.

Opening and saving the notebook

- Download the .ipynb file from Canvas or the course website
- Open it in JupyterLab, Jupyter Notebook, or Google Colab
- Save a personal copy before editing
- Keep code outputs and figures in the submitted file

Execution order matters

```
A = 1.20  
print(A)  
  
# A later cell can use A only after this cell runs.  
rms_component = A / np.sqrt(2)  
print(rms_component)
```

A restart clears stored variables. Run All rebuilds the notebook state from the first cell.

The new two-tone signal

```
A1, f1, phi1_deg = 1.20, 18.0, 25.0
A2, f2, phi2_deg = 0.45, 42.0, -15.0

def signal(t):
    return (A1*np.sin(2*np.pi*f1*t + np.deg2rad(phi1_deg))
            + A2*np.cos(2*np.pi*f2*t + np.deg2rad(phi2_deg)))
```

The function returns voltage values for any array of time values.

Meaning of the Python syntax

ARRAY OPERATIONS

`np.pi` supplies π .

`np.sin` and `np.cos` accept arrays.

`np.deg2rad` converts degrees to radians.

FUNCTION OUTPUT

`signal(t)` adds the two components at every requested time.

Changing `t` changes the sampling grid, not the physical signal definition.

Mean and RMS

MEAN

The average signed voltage over the record.

Each tone has zero mean over complete cycles.

RMS

A measure related to signal energy.

For distinct tones over complete cycles:

$$\text{RMS} = \sqrt{(A_1^2 + A_2^2)/2} = 0.9062 \text{ V}$$

Samples and the underlying waveform

DENSE REFERENCE

The gray curve in the notebook uses 20,001 calculated time points. It shows the mathematical signal clearly.

48 HZ SAMPLES

The blue markers contain only 10 measured values in the first 0.20 s. Lines between markers do not add information.

Sample rate, interval, and record length

```
fs = 126.0                # samples per second
dt = 1/fs                 # seconds between samples
duration = 1.00           # seconds
n = int(round(duration*fs))
t = np.arange(n)/fs

print(dt, n, t[-1])
```

For this one-second record, frequency-bin spacing is $\Delta f = 1/\text{duration} = 1$ Hz.

Nyquist frequency

- The Nyquist frequency equals $f_s/2$
- A sampled record can represent frequencies only from 0 to $f_s/2$
- The 42 Hz tone requires f_s greater than 84 Hz
- Rates of 336 and 126 Hz meet this condition. Rates of 72 and 48 Hz do not

Aliasing changes the measured frequency

FS = 126 HZ

Nyquist frequency: 63 Hz

Observed FFT peaks: 18 and 42 Hz

Both components remain below Nyquist.

FS = 48 HZ

Nyquist frequency: 24 Hz

Observed FFT peaks: 18 and 6 Hz

42 Hz appears at $|42 - 48| = 6$ Hz.

The sampling loop

```
sample_rates = [336.0, 126.0, 72.0, 48.0]
records = {}

for fs in sample_rates:
    n = int(round(duration*fs))
    t = np.arange(n)/fs
    x = signal(t)
    records[fs] = (t, x)
    print(fs, np.mean(x), np.sqrt(np.mean(x**2)))
```

A dictionary stores each time array and sampled signal under its sample rate.

Reading the time-domain figures

- Compare sample markers with the dense reference over the same time window
- Look for too few points per cycle and apparent changes in shape
- Read the printed mean and RMS values
- Avoid judging sample quality from smooth connecting lines

One-sided FFT calculation

```
n = len(x)
window = np.hanning(n)
coherent_gain = np.mean(window)

X = np.fft.rfft(x*window)
magnitude = np.abs(X)/(n*coherent_gain)
magnitude[1:-1] *= 2
frequency = np.fft.rfftfreq(n, d=1/fs)
```

The notebook applies a Hann window and corrects its amplitude loss.

Expected FFT peaks

ADEQUATE SAMPLE RATE

336 Hz gives peaks at 18 and 42 Hz.

126 Hz gives peaks at 18 and 42 Hz.

ALIASING

72 Hz gives peaks at 18 and 30 Hz.

48 Hz gives peaks at 18 and 6 Hz.

Guided notebook walkthrough

- 1 Define the signal**
- 2 Choose f_s and create sample times**
- 3 Plot samples beside the reference**
- 4 Compute the one-sided FFT**
- 5 Explain the physical meaning**

Required student responses

- Show the theoretical mean and RMS calculation
- Compare sampled mean and RMS values with theory
- List the two dominant frequencies for every sample rate
- Explain the aliased frequencies and connect the result to Lab 1

Common notebook problems

CODE PROBLEMS

NameError: run earlier cells.

Missing figure: run the plotting cell.

Changed variable: restart and Run All.

ANALYSIS PROBLEMS

Wrong units for phase.

Confusing f_s with signal frequency.

Calling a smooth line measured data.

Reporting peaks without explaining aliasing.

From the notebook to Moku:Go

- Choose a sample rate from the known or estimated sound frequency
- Record at least one second and save the data as a MAT file
- Plot the measured waveform and compute its spectrum
- Compare the dominant peak with the tuning-fork frequency and replay the recording

Pre-Lab 1 submission

BEFORE SUBMITTING

Run All cells.

Keep figures visible.

Complete four response sections.

Export to HTML or PDF if Canvas requests it.

DEADLINE

Normally posted Monday before the laboratory.

Submit before your own section begins.

Late pre-labs are not accepted.